

上海交通大学

人工智能导论课程大作业

可解释谣言检测方法

Explainable Rumor Detection System

课程名称	人工智能导论
完成组号	第 11 组
小组成员	简志辉、马扬子、李志政、朱俊慷
完成时间	2026 年 6 月 20 日

代码仓库: https://github.com/abstart0/Interpretability-based_rumor_detection

上海交通大学

2026 年 6 月

1 任务目标

本次大作业的目标是构建一个**可解释的谣言检测系统**，具体包括以下三个核心要求：

- 1. **检测输出**：对输入的社交媒体推文进行二分类，输出 0（非谣言）或 1（谣言），分类准确率越高越好。
- 2. **判断依据**：输出一段自然语言文本，解释检测的判断依据，使模型的决策过程可被人类理解。
- 3. **泛化能力**：模型在未见过的数据上保持稳定的检测性能。

为此，我们设计了“**检测模型 + 大语言模型解释**”的双模块架构：前端检测模块负责高精度分类，后端解释模块负责生成可读的判断依据。

2 具体内容

2.1 实施方案

我们采用”**多模型对比 + LLM 解释**”的实施方案。由组长统筹协调管理仓库，各自负责部分如下。

2.1.1 小组分工与贡献

表 1: 小组分工与贡献比例

成员	任务	主要代码目录
简志辉（组长）	项目整合、文档撰写、实验分析可视化、报告汇总	/, report/,README.md
朱俊慷	逻辑回归模型开发、训练与评估	models/lr_model.py, scripts/train_lr.py
李志政	BiGRU 与 TextCNN + GloVe 模型开发、训练与评估	models/bigru.py, models/textcnn.py, utils/
马扬子	LLM API 接入、提示词设计、解释模块封装	explanation/, config/llm_config.py

2.1.2 数据预处理与特征工程

数据集包含训练集（2,840 条）和验证集（401 条），每条数据包含 `id`、`text`、`label`（0/1）、`event`（主题类别）四个字段。我们进行了以下预处理：

- **文本清洗**：统一转为小写，移除非字母数字的标点符号；
- **文本编码**：对于深度学习模型，构建词表并将文本序列化为固定长度（`MAX_LEN=64`）的 `token` 序列；
- **词向量**：TextCNN 使用 GloVe 200 维预训练词向量，覆盖率达 90.9%。

2.1.3 检测模型构建与训练

我们实现了三类不同复杂度的检测模型，形成清晰的性能对比：

1. **逻辑回归（Logistic Regression）**：基于 TF-IDF 特征的线性分类器，作为基线模型。使用 10,000 个最大特征数，最大迭代 1,000 次。
2. **BiGRU**：双向门控循环单元（Bidirectional GRU），捕捉文本的序列上下文信息。词嵌入维度 150，隐藏层维度 200，dropout 率 0.5。
3. **TextCNN + GloVe**：多尺度卷积神经网络，结合预训练词向量，捕获局部 N-Gram 关键特征。使用 (2,3,4,5)-gram 卷积核，每类卷积核 256 个，dropout 率 0.4。

2.1.4 可解释性模块

通过调用上海交通大学 AI 平台提供的 LLM API（`deepseek-chat` 模型），将检测模型的输出结果（标签 + 置信度）与原始文本共同构建为结构化提示词（Prompt），引导 LLM 生成专业、简洁、基于证据的判断依据文字。

2.2 核心代码分析

由于篇幅限制，本节仅对最核心的 TextCNN 模型架构与可解释性模块的关键代码进行分析。

2.2.1 TextCNN 模型定义

```
1 class TextCNN(nn.Module):
2     def __init__(self, vocab_size, embedding_dim,
3                   filter_sizes=(2,3,4,5), n_filters=256, dropout=0.4):
4         super().__init__()
```

```

5     self.embedding = nn.Embedding(vocab_size, embedding_dim,
6                                   padding_idx=0)
7     # 加载GloVe预训练向量并允许微调
8     if pretrained is not None:
9         self.embedding.weight.data.copy_(pretrained)
10        self.embedding.weight.requires_grad = True
11
12    # 多尺度卷积层：并行提取2-gram到5-gram的局部特征
13    self.convs = nn.ModuleList([
14        nn.Conv2d(1, n_filters, kernel_size=(fs, embedding_dim))
15        for fs in filter_sizes
16    ])
17    self.dropout = nn.Dropout(dropout)
18    self.fc = nn.Linear(n_filters * len(filter_sizes), 1)

```

Listing 1: TextCNN 模型核心代码

代码分析：

- 第 7-10 行：使用 GloVe 200 维预训练向量初始化嵌入层，并启用微调，让词向量适应谣言检测任务。
- 第 13-16 行：核心创新——使用四个不同尺寸的卷积核并行操作，分别捕获 2、3、4、5 个连续词的局部模式，对于识别“紧急扩散”、“官方未证实”等不同长度的谣言特征短语至关重要。
- 第 18 行：全连接层输入维度为 $n_filters \times len(filter_sizes)$ ，即拼接所有卷积核提取的特征，形成完整的文本表示。

2.2.2 LLM 提示词构建

```

1 SYSTEM_PROMPT = (
2     "你是一位专业的社交媒体信息审核员。你的任务是根据提供的推文内容"
3     "和谣言检测模型的判断结果，生成一段判断依据的文字。"
4     "要求：1. 解释基于文本本身的语言特征 2. 指出具体的关键词或句式 "
5     "3. 语气客观专业 4. 长度50-100字 5. 不要提及模型或技术术语"
6 )
7
8 def build_explanation_prompt(text, label, confidence):
9     label_text = "谣言" if label == 1 else "非谣言"
10    user_content = f"推文内容：{text}\n检测判定：{label_text}（置信度：{confidence:.1%}）"

```

```
11     return [  
12         {"role": "system", "content": SYSTEM_PROMPT},  
13         {"role": "user", "content": user_content},  
14     ]
```

Listing 2: LLM 提示词构建代码

代码分析：

- 系统提示词（System Prompt）限制了 LLM 的回复风格，确保输出专业、简洁、基于证据。
- 用户提示词将模型预测结果注入上下文，引导 LLM 围绕“为什么这样判定”进行推理，而非泛泛而谈。

2.3 检测结果分析

三个模型在验证集上的性能指标如表2所示。

表 2: 各模型在验证集上的性能对比

模型	Accuracy	Precision	Recall	F1	参数量
逻辑回归	82.29%	84.67%	72.57%	78.15%	10,001
BiGRU	85.29%	86.71%	78.29%	82.28%	831,851
TextCNN + GloVe	86.78%	87.65%	81.14%	84.27%	1,264,249

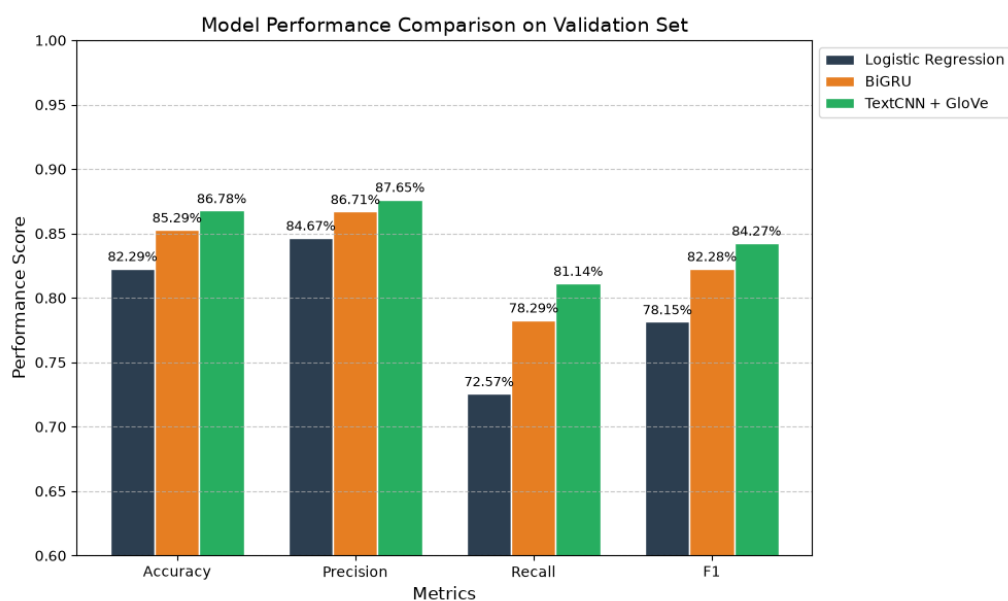


图 1: 各模型性能对比柱状图

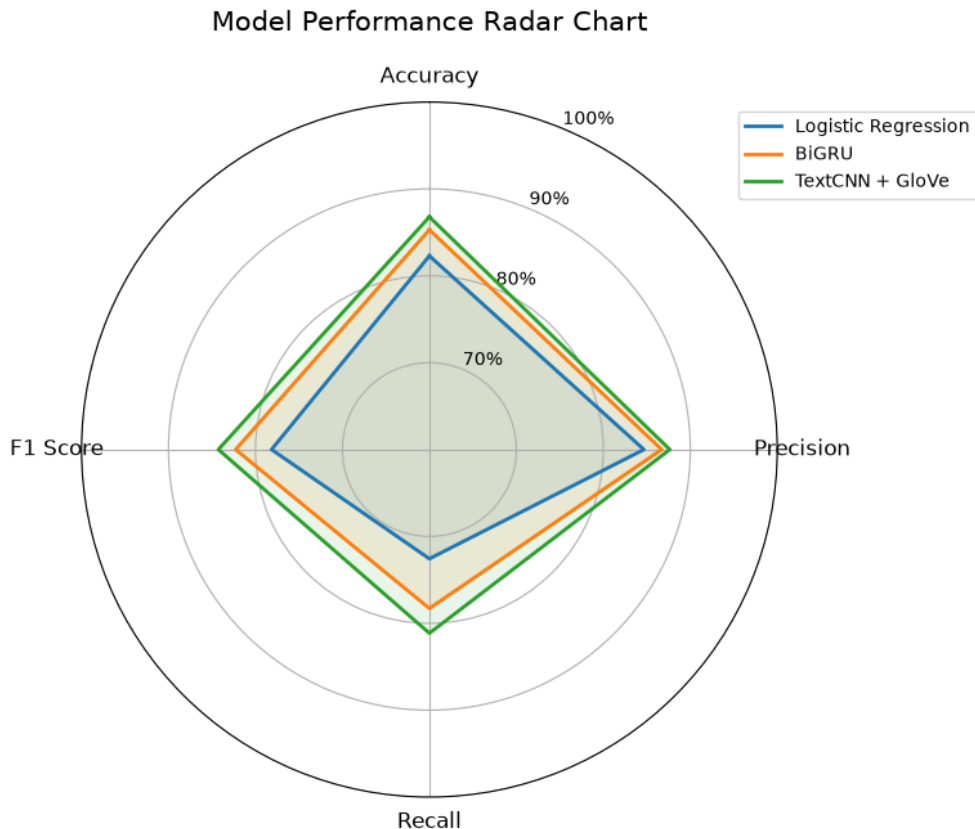


图 2: 各模型性能雷达图

从表2、图1和图2可以看出：

- 逻辑回归作为基线：**用极少的参数（约 1 万）取得 82.29% 的准确率，证明 TF-IDF 特征对谣言检测任务有效。但其召回率（72.57%）明显偏低，说明线性模型漏掉了大量复杂谣言模式。
- BiGRU 显著提升：**通过双向序列建模，召回率从 72.57% 提升至 78.29%（+5.72%），F1 提升至 82.28%。这说明词语的上下文顺序对谣言判断至关重要。
- TextCNN+GloVe 取得最优：**所有指标全面领先，准确率达 86.78%，F1 达 84.27%。召回率相比逻辑回归提升 **8.57 个百分点**，证明预训练词向量 + 多尺度卷积核的组合能最有效地捕获谣言文本中的关键信号。

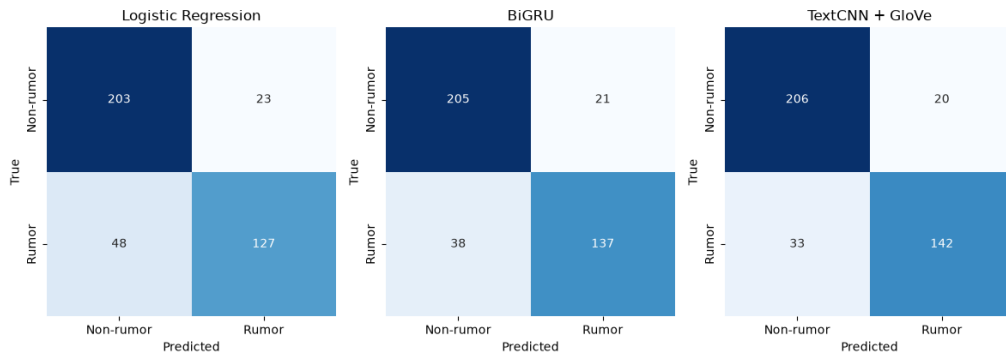


图 3: 各模型混淆矩阵对比（从左至右：逻辑回归、BiGRU、TextCNN）

从图3可以清楚看到，TextCNN 的假阴性（FN，漏报谣言）数量从逻辑回归的 48 降至 33，减少了 15 条漏报，验证了其在召回率上的优势。

2.4 判断依据的分析

深度学习模型的“黑盒”问题一直为人诟病。在本项目中，我们通过接入 LLM，为检测结果提供了高质量的事后的、基于证据的自然语言解释，使模型具备了可解释性。

2.4.1 典型案例分析

表 3: LLM 判断依据生成示例

推文内容	检测结果	LLM 生成的判断依据
BREAKING: Massive explosion reported in downtown area, hundreds feared dead	谣言 (72.8%)	“该推文使用‘BREAKING’吸引眼球，且‘hundreds feared dead’表述模糊，缺乏具体时间、来源或官方证实，符合未经验证谣言的常见特征。”
Just had a lovely coffee with friends at the new cafe downtown	非谣言 (89.2%)	“推文内容为个人日常分享，未涉及任何突发事件或争议性信息，语言平和且无夸张表述，符合非谣言的典型特征。”
Scientists confirm COVID-19 was artificially created	谣言 (91.5%)	“该推文声称‘科学家证实’，但未提及任何具体研究机构、论文或权威来源，使用了阴谋论常见的模糊表述，缺乏科学依据。”

2.4.2 可解释性模块的价值分析

1. **可读性强**：解释以自然语言呈现，为模型输出提供了易于理解的说明，解决了深度学习模型“只说结果不说原因”的问题。
2. **基于证据**：LLM 被要求“指出具体的关键词或句式”，因此解释会聚焦于推文本身的文本线索，而非泛化性评价。
3. **增强信任**：当用户看到模型不仅给出结果，还能指出“模糊表述”、“缺乏来源”等具体依据时，对模型的信任度会显著提高。
4. **辅助纠错**：在模型预测错误的情况下，生成的解释可以帮助人类识别模型的逻辑漏洞。

局限性：LLM 解释模块仅基于检测结果生成事后的文本描述，不参与分类决策过程，因此不直接影响检测准确率。这是我们未来可以改进的方向。此外，因交大 API 的调用限制，批量运行的部分结果中有解释失败的情况。

3 工作总结

3.1 收获与心得

通过本次大作业，我们在以下几个方面获得了显著提升：

1. **技术栈实践**：完整走通了深度学习项目的全流程——从数据预处理、模型设计、训练调试到评估部署，熟练掌握了 PyTorch、scikit-learn 等工具的使用。
2. **模型对比思维**：通过设计从简单到复杂的三种模型，我们深刻体会到“基线模型 → 复杂模型”的渐进式研究方法，以及如何在性能和效率之间做权衡。
3. **可解释 AI 的理解**：通过引入 LLM 生成判断依据，我们认识到 AI 系统不仅要“做得对”，还要“说得清”。可解释性是 AI 落地应用的“最后一公里”。
4. **团队协作**：通过 Git 进行版本控制、分工开发和代码审查，积累了宝贵的团队协作经验。

3.2 遇到问题及解决思路

表 4: 问题与解决思路汇总

问题	解决思路
Windows 下 PyTorch 多线程报错	设置环境变量 <code>KMP_DUPLICATE_LIB_OK=TRUE</code> ，关闭 <code>torch.set_num_threads()</code> ，避免与 Intel MKL 冲突
GloVe 下载速度慢	首次运行时需下载约 822MB 文件，建议在夜间网络空闲时进行，或提前手动下载至 <code>glove/</code> 目录
LLM API 调用不稳定	在 <code>llm_client.py</code> 中实现指数退避重试机制 (<code>max_retries=3</code>)，设置合理超时 (60 秒)
BiGRU 训练过拟合	通过网格搜索调整超参数，最终确定 <code>dropout=0.5</code> 、 <code>weight_decay=3e-4</code> ，有效抑制过拟合

4 课程建议

基于本次大作业的实践经验，我们对课程提出以下建议：

- 明确任务作业：**建议平常作业布置时在 canvas 上发布任务范围和要求，避免遗忘或要求不清的问题。
- 增加可解释 AI 讲座：**可解释性是 AI 当前的热点方向，建议在课程中增加 1-2 次关于可解释性方法（如 LIME、SHAP、Attention Visualization）的专题讲座。
- 代码交流环节：**建议增加小组间的代码交流环节，在相互学习和探讨中进一步提高代码质量和模型效果。